

Lecture 11

Parallel Processing

Introduction

- **Parallel computing** is a form of computation in which many calculations are carried out simultaneously, operating on the principle that large problems can often be divided into smaller ones, which are then solved concurrently ("in parallel").
- There are several different forms of parallel computing: **bit-level**, **instruction level**, **data level**, and **task level parallelism**.
- Parallelism appears in various forms, such as lookahead, pipelining, vectorization, concurrency, simultaneity, data parallelism, partitioning, interleaving, overlapping, multiplicity, replication, time sharing, space sharing, multitasking, multiprogramming, multithreading, and distributed computing at different processing levels.

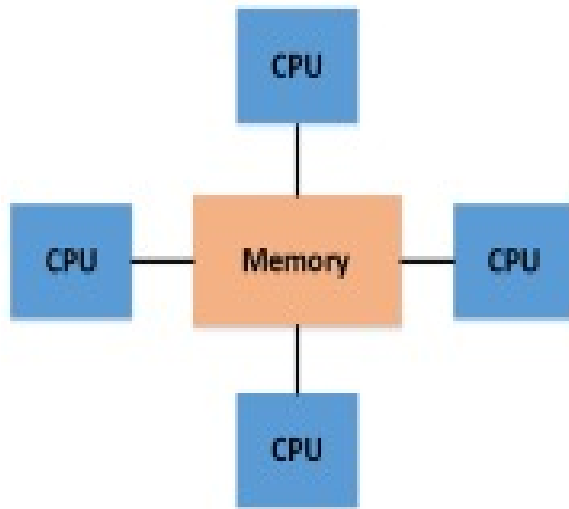
Flynn's taxonomy

	Single instruction	Multiple instruction
Single data	<u>SISD</u>	<u>MISD</u>
Multiple data	<u>SIMD</u>	<u>MIMD</u>

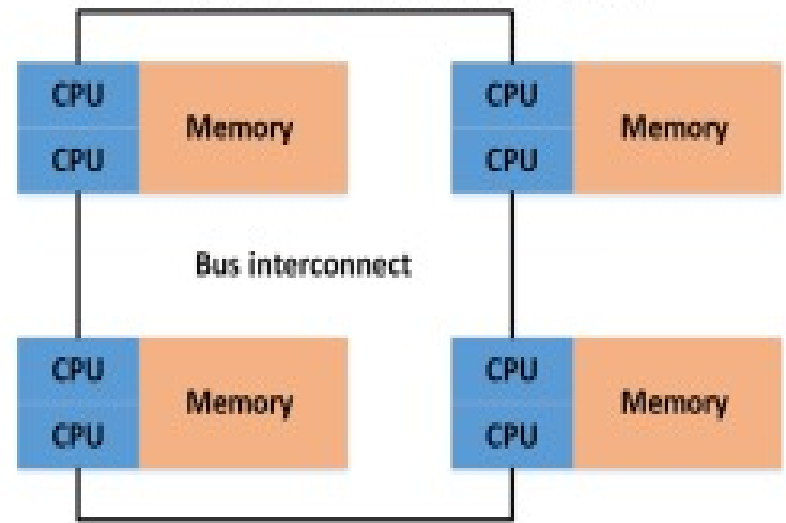
Multiprocessor and Multicomputer

- Two categories of parallel computers are architecturally distinguished by having a shared common memory or distributed memories.
- Computer architectures in which each element of main memory can be accessed with equal **latency** and **bandwidth** are known as **Uniform Memory Access** (UMA) systems.
- Typically, that can be achieved only by a **shared memory** system, in which the memory is not physically distributed.
- A system that does not have this property is known as a **Non-Uniform Memory Access** (NUMA) architecture. Distributed memory systems have non-uniform memory access.

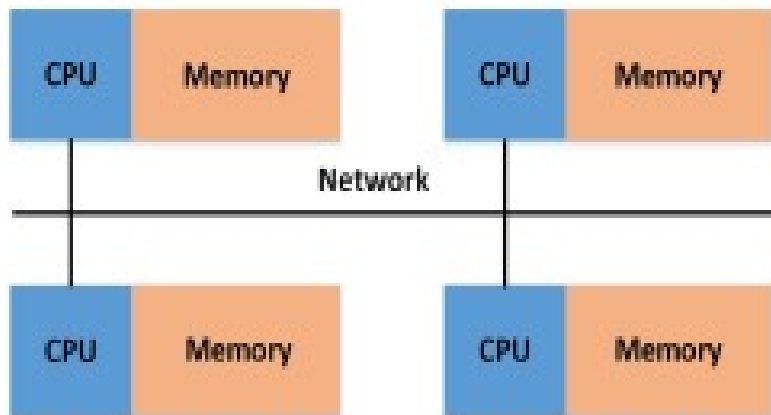
Shared memory (UMA)



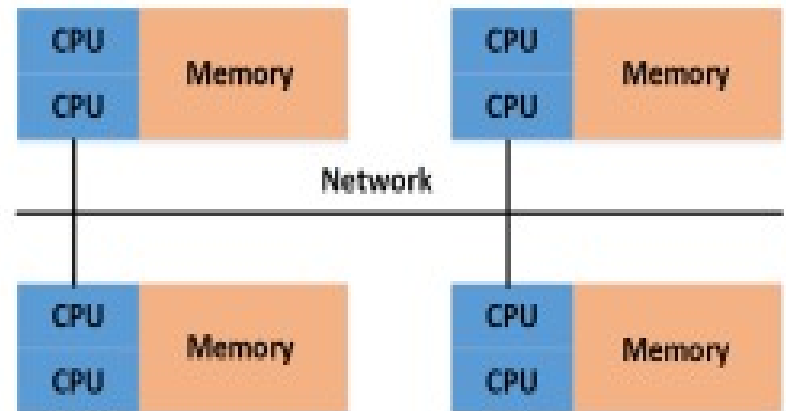
Shared memory (NUMA)



Distributed memory

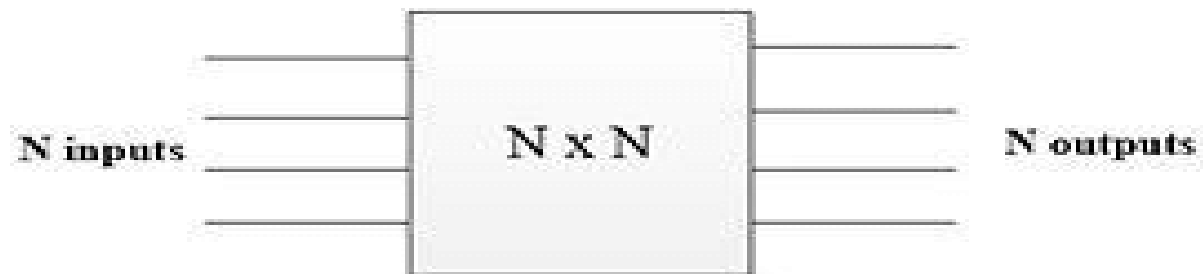
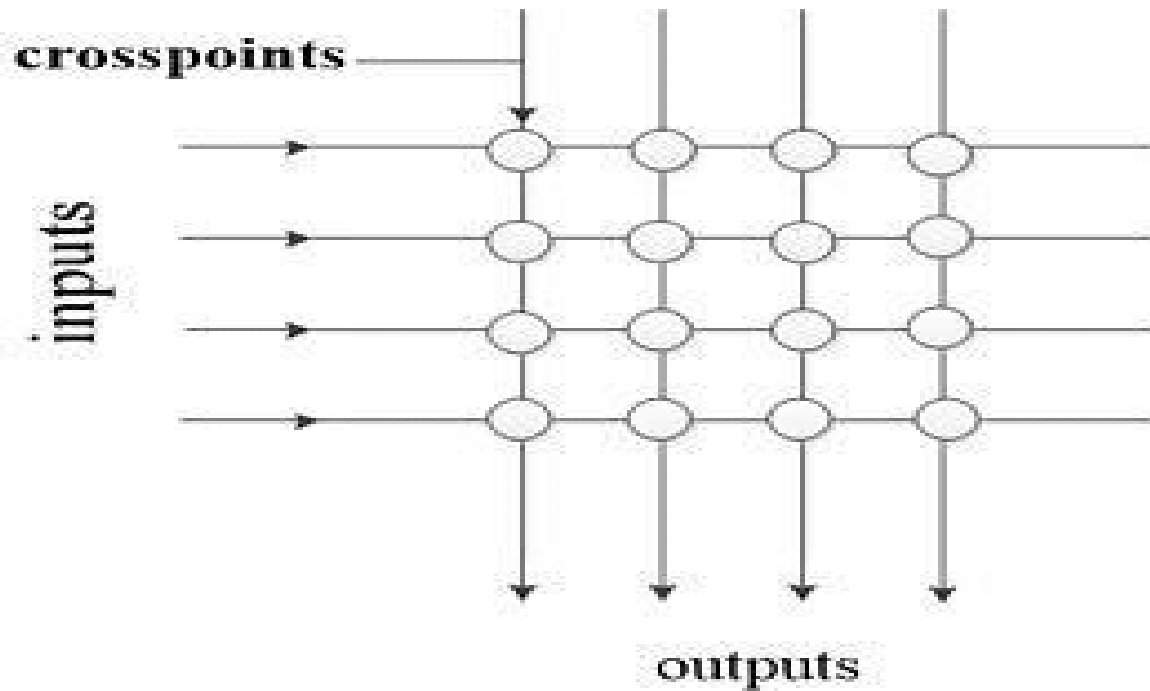


Hybrid memory



Interconnection Network

- Processor–processor and processor–memory communication can be implemented in hardware in several ways, including via shared (either multiported or **multiplexed**) memory, a **crossbar switch**, a shared **bus** or an interconnect network of a myriad of **topologies** including **star**, **ring**, **tree**, **hypercube**, fat hypercube (a hypercube with more than one processor at a node), or **n-dimensional mesh**.
- Parallel computers based on interconnect networks need to have some kind of **routing** to enable the passing of messages between nodes that are not directly connected.
- The medium used for communication between the processors is likely to be hierarchical in large multiprocessor machines.



Crossbar Switch

Operational models used in programming multiprocessor systems

- **Multiprogramming.**

- Traditionally, multiprogramming is defined as multiple independent programs running on a single processor or on a multiprocessor by time-sharing use of the system resources.
- A multiprogrammed multiprocessor allows multiple programs to run concurrently through time sharing of all processors in the system. Multiple programs are interleaved in their CPU and I/O activities.
- When a program enters I/O mode, the processor switches to another program.

- **Multiprocessing.**

- When multiprogramming is implemented at the process level on a multiprocessor, it is called multiprocessing.
- Two types of multiprocessing are specified based on the interprocessor communications: MIMD, and MPMD.

Cont.,

- **Multitasking.**
 - A single program can be partitioned into multiple interrelated tasks concurrently executed on a multiprocessor.
 - Thus multitasking provides the parallel execution of two or more parts of a single program.
- **Multithreading.**
 - The traditional operating system (OS) has a single-threaded kernel in which only one process can receive OS kernel service at a time.
 - Hardware multithreading allows multiple threads to share the functional units in a single processor in an overlapping fashion.
 - In multiprocessor, it is needed to extend the single kernel to be multithreaded to allow multiple threads of lightweight processes to share the same address space and to be executed by the same or different processors simultaneously.
 - Each thread may use a separate program counter.
 - Resource conflicts are the major problem to be resolved in a multithreaded architecture.

Types of parallelism

- [a] Bit-level parallelism

- From the advent of **very-large-scale integration** (VLSI) computer-chip fabrication technology in the 1970s until about 1986, speed-up in computer architecture was driven by doubling **computer word size**—the amount of information the processor can manipulate per cycle.
- Increasing the word size reduces the number of instructions the processor must execute to perform an operation on variables whose sizes are greater than the length of the word.
- For example, where an **8-bit** processor must **add** two **16-bit integers**, the processor must first add the 8 lower-order bits from each integer using the standard addition instruction, then add the 8 higher-order bits using an add-with-carry instruction and the **carry bit** from the lower order addition; thus, an 8-bit processor requires two instructions to complete a single operation, where a 16-bit processor would be able to complete the operation with a single instruction.

Cont.,

- **[b] Instruction-level parallelism (Superscalar Processor)**

- A computer program is, in essence, a stream of instructions executed by a processor.
- These instructions can be **re-ordered** and combined into groups, only if there is no **data dependency** between them, which are then executed in parallel without changing the result of the program.
- This is known as instruction-level parallelism.
- A superscalar **CPU** architecture implements a form of **instruction level parallelism** within a single processor.
- It therefore allows faster CPU **throughput** than would otherwise be possible at a given **clock rate**.
- A superscalar processor (VLIW) executes more than one instruction during a clock cycle by simultaneously dispatching multiple instructions to redundant functional units on the processor.
- Each functional unit is not a separate CPU core but an execution resource within a single CPU such as an **arithmetic logic unit**, a bit shifter, or a **multiplier**.

Cont.,

- [c] Task parallelism

- **Task parallelism** (also known as **function parallelism** and **control parallelism**) is a form of **parallelization** of computer code across multiple **processors** in **parallel computing** environments.
- Task parallelism focuses on distributing execution processes (threads) across different parallel computing nodes.
- In a multiprocessor system, task parallelism is achieved when each processor executes a different thread (or process) on the same or different data.
- The threads may execute the same or different code.
- In the general case, different execution threads communicate with one another as they work.
- Communication usually takes place by passing data from one thread to the next as part of a **workflow**.
- As a simple example, if we are running code on a 2-processor system (**CPUs** "a" & "b") in a **parallel** environment and we wish to do tasks "A" and "B", it is possible to tell CPU "a" to do task "A" and CPU "b" to do task "B" simultaneously, thereby reducing the **run time** of the execution.

Cont.,

- [D] Data Parallelism

- **Data parallelism** is a form of parallelization of computing across multiple **processors** in **parallel computing** environments.
- Data parallelism focuses on distributing the data across different parallel computing nodes.
- In a multiprocessor system executing a single set of instructions (**SIMD**), data parallelism is achieved when each processor performs the same task on different pieces of distributed data.
- In some situations, a single execution thread controls operations on all pieces of data.
- In others, different threads control the operation, but they execute the same code.

Classes of parallel computers

- **[1] Multicore computer**

- A multicore computer, also known as a **chip multiprocessor**, combines two or more processors (called cores) on a single piece of silicon (called die).
- Typically, each core consists of all components on independent processor, such as registers, ALU, pipeline hardware, and control unit.
- These processors differ from **superscalar** processors, which can issue multiple instructions per cycle from one instruction stream (thread); in contrast, a multicore processor can issue multiple instructions per cycle from multiple instruction streams.
- Each core in a multicore processor can potentially be superscalar as well—that is, on every cycle, each core can issue multiple instructions from one instruction stream.
- **Simultaneous multithreading** (of which Intel's **Hyperthreading** is the best known) was an early form of pseudo-multicoreism.
- A processor capable of simultaneous multithreading has only one execution unit ("core"), but when that execution unit is idling (such as during a **cache miss**), it uses that execution unit to process a second thread.

Cont.,

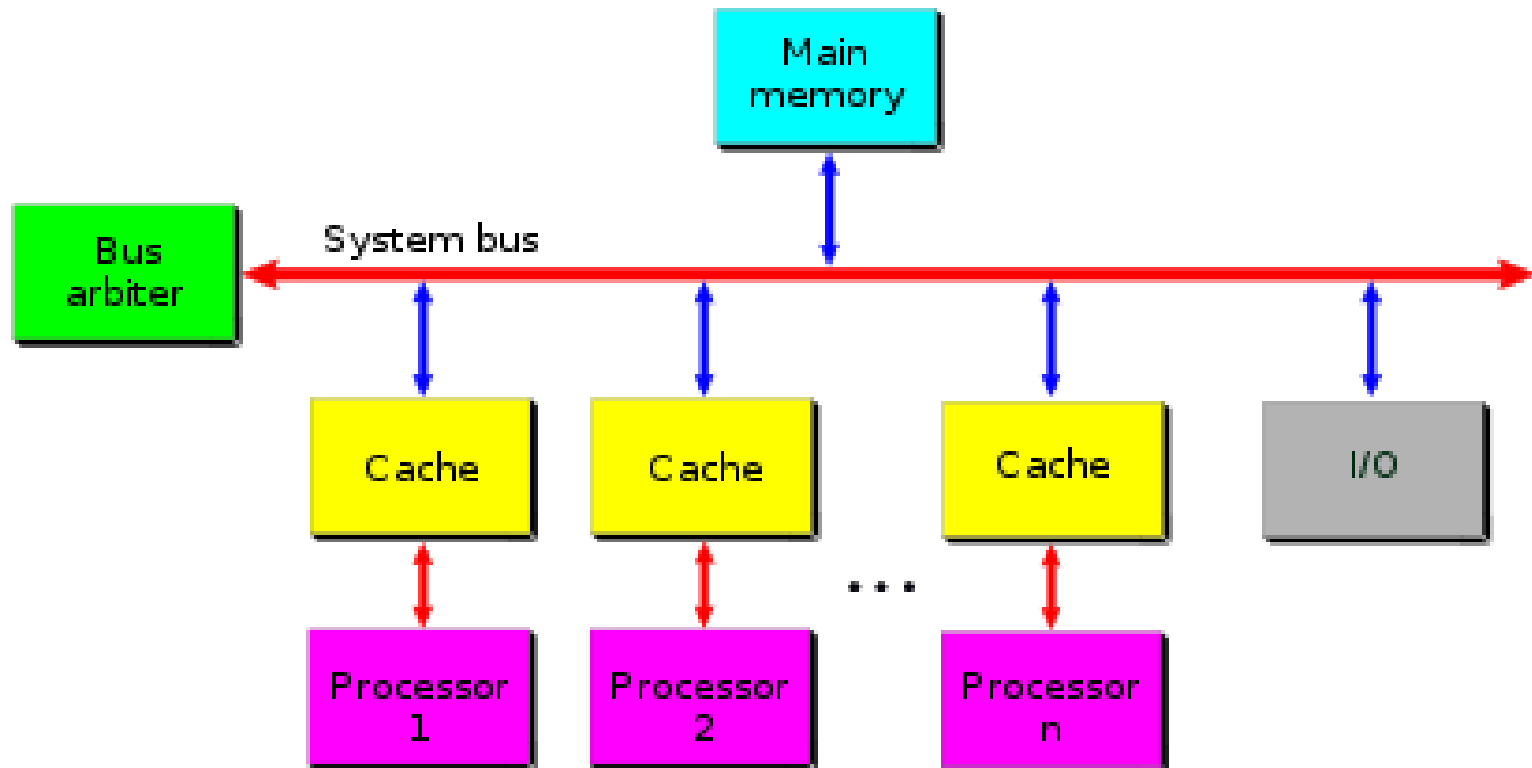
- Simultaneous multithreading (SMT) is one of the two main implementations of multithreading, the other form being temporal multithreading.
- In temporal multithreading, only one thread of instructions can execute in any given pipeline stage at a time.
- In simultaneous multithreading, instructions from more than one thread can be executing in any given pipeline stage at a time.
- This is done without great changes to the basic processor architecture.
- The main additions needed are the ability to fetch instructions from multiple threads in a cycle, and a larger register file to hold data from multiple threads.
- The number of concurrent threads can be decided by the chip designers, but practical restrictions on chip complexity have limited the number to two for most SMT implementations, though there have been as many as 8 threads per core.

Cont.

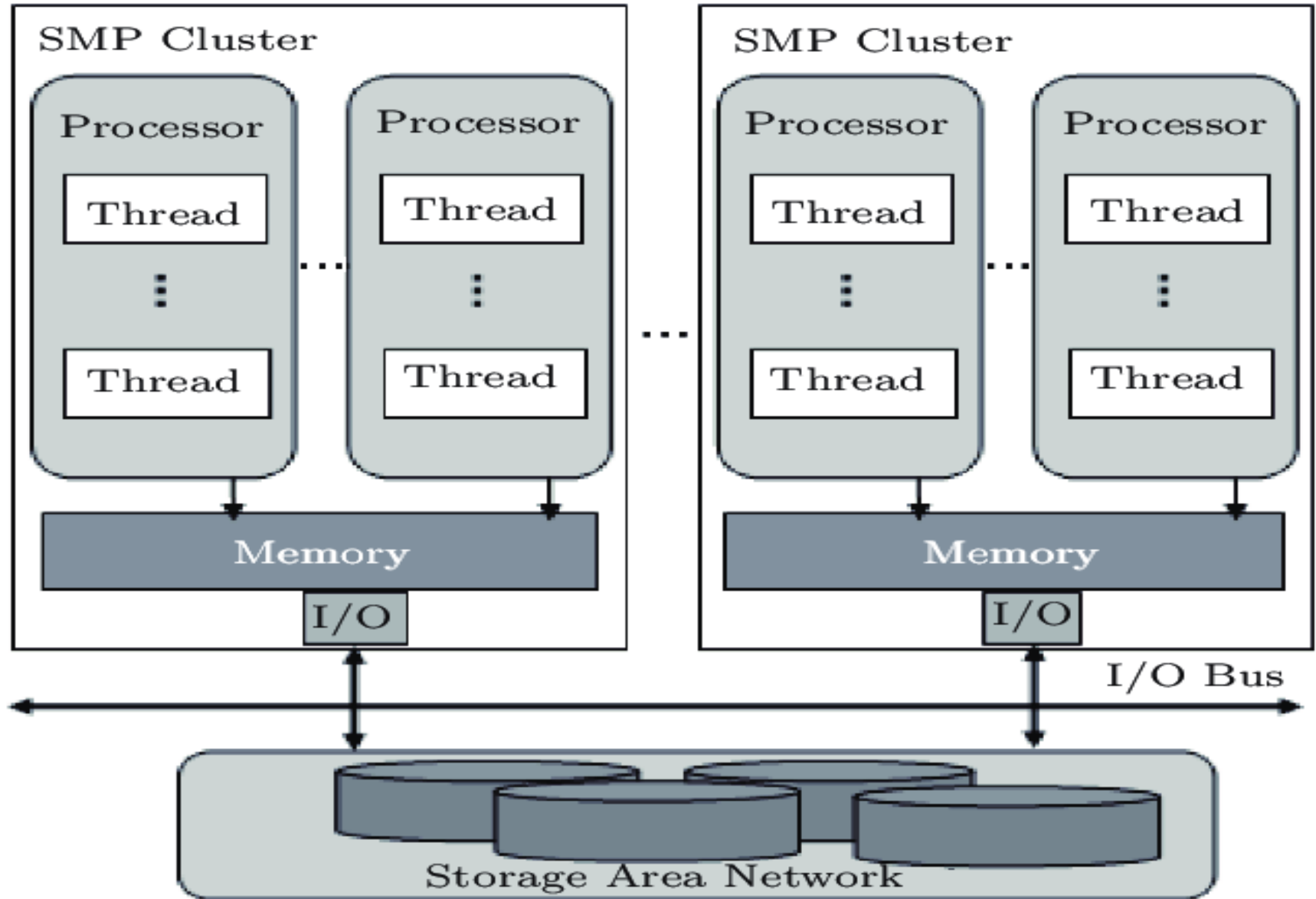
- **[2] Symmetric multiprocessing**

- A symmetric multiprocessor (SMP) is a computer system with multiple identical processors that share the same memory and connect via a bus.
- SMP systems are *tightly coupled multiprocessor systems* with a pool of homogeneous processors running independently, each processor executing different programs and working on different data and with capability of sharing common resources (memory, I/O device, interrupt system and so on) and connected using a *system bus* or a *crossbar*.
- Most common multiprocessor systems today use an SMP architecture.
- In the case of *multi-core* processors, the SMP architecture applies to the cores, treating them as separate processors.

SMP - symmetric multiprocessor system



MPP Architecture



MOTIVATIONS BEHIND HETEROGENEOUS CMP

- Homogenous CMP usually consists of several numbers of processors of same characteristics like same issue width, cache size, clock frequency etc.
- The processor is designed with moderate number of all resources as it has to cope up with both complex sequential programs with low thread level parallelism (TLP), or simple program with high TLPs.
- Homogenous CMPs resources are wasted when application with high TLPs is used.
- On the other hand, Heterogeneous CMPs consist of different type of processors in a single chip.
- Processors can vary from higher level detail such as instruction set or, micro-architectural detail such as clock frequency, voltage level, issue width, and cache size.

Software for Parallel Programming

- **Parallel computer programs** are more difficult to write than sequential ones, because concurrency introduces several new classes of potential **software bugs**, of which **race conditions** are the most common.
- **Communication** and **synchronization** between the different subtasks are typically one of the greatest obstacles to getting good parallel program performance.

Dependencies

- Understanding **data dependencies** is fundamental in implementing **parallel algorithms**.
- No program can run more quickly than the longest chain of dependent calculations (known as the **critical path**), since calculations that depend upon prior calculations in the chain must be executed in order.
- However, most algorithms do not consist of just a long chain of dependent calculations; there are usually opportunities to execute independent calculations in parallel.

Parallel programming languages

- Concurrent programming languages, libraries, APIs, and parallel programming models (such as Algorithmic Skeletons) have been created for programming parallel computers.
- These can generally be divided into classes based on the assumptions they make about the underlying memory architecture—shared memory, distributed memory, or shared distributed memory.
- Shared memory programming languages communicate by manipulating shared memory variables.
- Distributed memory uses message passing.
- POSIX Threads and OpenMP are two of most widely used shared memory APIs, whereas Message Passing Interface (MPI) is the most widely used message-passing system API.

Cont.,

- An **application programming interface (API)** is a [protocol](#) intended to be used as an [interface](#) by [software components](#) to communicate with each other.
- An API is a library that may include specification for [routines](#), [data structures](#), [object classes](#), and variables.
- An API specification can take many forms, including an International Standard such as [POSIX](#), vendor documentation such as the [Microsoft Windows API](#), the [libraries](#) of a programming language, e.g., [Standard Template Library](#) in C++ or [Java API](#).

Automatic parallelization

- Automatic parallelization of a sequential program by a **compiler** is the **holy grail** of parallel computing.
- Despite decades of work by compiler researchers, automatic parallelization has had only limited success.
- Mainstream parallel programming languages remain either **explicitly parallel** or (at best) **partially implicit**, in which a programmer gives the compiler **directives** for parallelization.
- A few fully implicit parallel programming languages exist—**SISAL**, Parallel **Haskell**, and (for **FPGAs**) **System C**, **VHDL**, and **Verilog**.
- Sisal (Streams and Iteration in a Single Assignment Language) is one of the true pioneers of high-level parallel programming.

Race conditions

- Race conditions arise in software when separate computer **processes** or **threads** of execution depend on some shared state.
- Operations upon shared states are **critical sections** that must be **mutually exclusive**.
- This protected section is the critical section or critical region.
- It cannot be executed by more than one process at a time.
- Failure to obey this rule opens up the possibility of corrupting the shared state.
- It is therefore better to avoid race conditions by careful software design rather than attempting to fix them afterwards.

Example

Thread A		Thread B
1A: Read variable V		1B: Read variable V
2A: Add 1 to variable V		2B: Add 1 to variable V
3A Write back to variable V		3B: Write back to variable V

Cont.,

- If instruction 1B is executed between 1A and 3A, or if instruction 1A is executed between 1B and 3B, the program will produce incorrect data.
- This is known as a **race condition**.
- The programmer must use a **lock** to provide **mutual exclusion**.
- A lock is a programming language construct that allows one thread to take control of a variable and prevent other threads from reading or writing it, until that variable is unlocked.
- The thread holding the lock is free to execute its **critical section** (the section of a program that requires exclusive access to some variable), and to unlock the data when it is finished.
- Therefore, to guarantee correct program execution, the above program can be rewritten to use locks:

Thread A	Thread B
1A: Lock variable V	1B: Lock variable V
2A: Read variable V	2B: Read variable V
3A: Add 1 to variable V	3B: Add 1 to variable V
4A Write back to variable V	4B: Write back to variable V
5A: Unlock variable V	5B: Unlock variable V

Cont.,

- Where, One thread will successfully lock variable V, while the other thread will be **locked out**—unable to proceed until V is unlocked again. This guarantees correct execution of the program. Locks, while necessary to ensure correct program execution, can greatly slow a program.

CONCLUSION

- In processor design, there are several ways to improve the performance of the CPU: one is [superscalar](#) technique which tries to increase Instruction Level Parallelism (ILP), the other is [multithreading](#) approach exploiting Thread Level Parallelism (TLP).
- Superscalar means executing multiple instructions at the same time while chip-level multithreading (CMT) executes instructions from multiple threads within one processor chip at the same time.